MIDTERM REPORT

Algorithm Design and Analysis (G)

Student:

Hisyam Syafa Raditya / 5025241130

INFORMATICS ENGINEERING DEPARTMENT

FACULTY OF INTELLIGENT ELECTRICAL AND INFORMATICS TECHNOLOGY

INSTITUT TEKNOLOGI SEPULUH NOPEMBER

Surabaya, 26 March 2026

Contents

1 Problem 1 — `isBST()` [20 points] 3
1.1 Phase 1: Understand . . . 3
1.2 Phase 2: Abstract . . . 3
1.3 Phase 3: Design . . . 3
1.4 Phase 4: Analyze . . . 3
1.5 Phase 5: Implement . . . 4

2 Problem 2 — `printDescending()` [10 points] 5
2.1 Phase 1: Understand . . . 5
2.2 Phase 2: Abstract . . . 5
2.3 Phase 3: Design . . . 5
2.4 Phase 4: Analyze . . . 5
2.5 Phase 5: Implement . . . 5

3 Problem 3 — `max()` [10 points] 6
3.1 Phase 1: Understand . . . 6
3.2 Phase 2: Abstract . . . 6
3.3 Phase 3: Design . . . 6
3.4 Phase 4: Analyze . . . 6
3.5 Phase 5: Implement . . . 6

4 Problem 4 — `isHeightBalanced()` [10 points] 7
4.1 Phase 1: Understand . . . 7
4.2 Phase 2: Abstract . . . 7
4.3 Phase 3: Design . . . 7
4.4 Phase 4: Analyze . . . 7
4.5 Phase 5: Implement . . . 7

5 Problem 5 — `insertHB()` [10 points] 9
5.1 Phase 1: Understand . . . 9
5.2 Phase 2: Abstract . . . 9
5.3 Phase 3: Design . . . 9
5.4 Phase 4: Analyze . . . 9
5.5 Phase 5: Implement . . . 9

6 Problem 6 — `rebalanceForLeft()` [15 points] 11
6.1 Phase 1: Understand . . . 11
6.2 Phase 2: Abstract . . . 11
6.3 Phase 3: Design . . . 11
6.4 Phase 4: Analyze . . . 12
6.5 Phase 5: Implement . . . 12

7 Problem 7 — `rebalanceForRight()` [15 points] 13
7.1 Phase 1: Understand . . . 13
7.2 Phase 2: Abstract . . . 13
7.3 Phase 3: Design . . . 13
7.4 Phase 4: Analyze . . . 14

7.5 Phase 5: Implement 14

8 Problem 8 — `deleteHB()` [10 points] 15

8.1 Phase 1: Understand 15
8.2 Phase 2: Abstract 15
8.3 Phase 3: Design 16
8.4 Phase 4: Analyze 16
8.5 Phase 5: Implement 16

Complexity Summary 18

Academic Integrity Pledge 19

1 Problem 1 — `isBST()` [20 points]

1.1 Phase 1: Understand

The task is to verify whether a given binary tree satisfies the Binary Search Tree (BST) property. In a valid BST, every node's value must be strictly greater than all values in its left subtree and less than or equal to all values in its right subtree. The public interface delegates to a private recursive helper that carries range constraints down the tree:

```
public static boolean isBST(MyTree t)
private static boolean isBST(MyTree t, int lowerBound, int
   upperBound)
```

1.2 Phase 2: Abstract

A common mistake is to compare only adjacent parent-child pairs. This fails for cases such as a node whose value violates an ancestor's constraint even though it satisfies the local parent constraint. The correct approach is to propagate a valid range [*lowerBound*, *upperBound*] through every recursive call.

The algorithm follows a *range-constrained recursive validation* pattern:

- An empty tree is trivially a BST (base case).
- If the current node's value falls outside the allowed range, return `false`.
- When descending *left*, the current value becomes the new upper bound (left children must be less than or equal to the parent's value).
- When descending *right*, the current value plus one becomes the new lower bound (right children must be strictly greater).

An integer overflow edge case arises when the current value equals `Integer.MAX_VALUE`: incrementing it to set the right-subtree lower bound would wrap around. This case is handled by requiring the right subtree to be empty instead.

1.3 Phase 3: Design

Algorithm 1 isBST(t, lo, hi)

```
if t is empty then return true
end if
if t.value < lo or t.value > hi then return false
end if
if t.value = INT_MAX then
    rightOK ← t.right is empty
else
    rightOK ← isBST(t.right, t.value + 1, hi)
end ifreturn isBST(t.left, lo, t.value) and rightOK
```

1.4 Phase 4: Analyze

- **Time complexity:** $O(n)$ — every node is visited exactly once.

- **Space complexity:** $O(h)$ — the recursion stack grows proportionally to the tree height h; worst case $O(n)$ for a degenerate (fully skewed) tree.
- **Correctness:** The range propagation mechanism correctly enforces the global BST invariant, not just local parent-to-child constraints. The integer-overflow edge case is explicitly handled.

1.5 Phase 5: Implement

```java
public static boolean isBST(MyTree t) {
    return isBST(t, Integer.MIN_VALUE, Integer.MAX_VALUE);
}

private static boolean isBST(MyTree t, int lowerBound, int upperBound) {
    if (t.getEmpty()) {
        return true;
    }
    if (t.getValue() < lowerBound || t.getValue() > upperBound) {
        return false;
    }
    boolean isRightValid = t.getValue() == Integer.MAX_VALUE ?
                           t.getRight().getEmpty() :
                           isBST(t.getRight(), t.getValue() + 1, upperBound);
    return isBST(t.getLeft(), lowerBound, t.getValue()) && isRightValid;
}
```

Listing 1: isBST() in DAA1.java

2 Problem 2 — `printDescending()` [10 points]

2.1 Phase 1: Understand

Print all values stored in a BST in descending (largest-to-smallest) order using a purely recursive function. No auxiliary list or array may be used; the ordering must arise from the traversal itself.

```
public static void printDescending(MyTree t)
```

2.2 Phase 2: Abstract

A standard in-order traversal (left → root → right) of a BST yields values in ascending order. Reversing the visit sequence to right → root → left produces the values in descending order. This is referred to as a *reverse in-order traversal*.

Because values are printed directly as each node is visited, the call stack itself serves as the implicit ordering mechanism. No list is created at any point.

2.3 Phase 3: Design

Algorithm 2 PRINTDESCENDING(t)

1: **if** t is empty **then return**
2: **end if**
3: PRINTDESCENDING(t.right)
4: **print** t.value
5: PRINTDESCENDING(t.left)

2.4 Phase 4: Analyze

- **Time complexity:** $O(n)$ — each node is visited exactly once.
- **Space complexity:** $O(h)$ — depth of the recursion stack; no list is allocated.
- **Correctness:** The BST invariant guarantees that values in the right subtree are always $\geq$ the root and values in the left subtree are $\leq$ the root. Visiting right first therefore always yields a non-increasing sequence.

2.5 Phase 5: Implement

```java
public static void printDescending(MyTree t) {
    if (t.getEmpty()) {
        return;
    }
    printDescending(t.getRight());
    System.out.print(t.getValue() + " ");
    printDescending(t.getLeft());
}
```

Listing 2: printDescending() in DAA1.java

3 Problem 3 — `max()` [10 points]

3.1 Phase 1: Understand

Find the maximum value in a non-empty BST. The implementation must run in $O(\log n)$ for a balanced BST, traverse only a single path (never visiting the left subtree), and must not compare values of sibling nodes.

```
public static int max(MyTree t)
```

3.2 Phase 2: Abstract

In a BST, larger values are always placed to the *right*. As a consequence, the maximum element is always located at the *rightmost* node in the tree. No comparison between sibling subtrees is ever necessary. The algorithm is a simple rightmost-path descent: follow right children until there is no right child, then return the current value.

3.3 Phase 3: Design

Algorithm 3 MAX(t)

1: **if** t is empty **then throw** `IllegalArgumentException`
2: **end if**
3: **if** t.right is empty **then return** t.value
4: **end ifreturn** MAX(t.right)

3.4 Phase 4: Analyze

- **Time complexity:** $O(h)$, where h is the height of the tree. For a height-balanced BST, $h = O(\log n)$, meeting the stated efficiency requirement.
- **Space complexity:** $O(h)$ recursion stack frames.
- **Correctness:** The function never inspects the left subtree and makes no direct value comparisons between siblings — the decision to continue descending is purely structural ("does a right child exist?"). An empty tree input is explicitly rejected with an exception.

3.5 Phase 5: Implement

```java
public static int max(MyTree t) {
    if (t.getEmpty()) {
        throw new IllegalArgumentException("Tree is empty");
    }
    if (t.getRight().getEmpty()) {
        return t.getValue();
    }
    return max(t.getRight());
}
```

Listing 3: max() in DAA1.java

4 Problem 4 — `isHeightBalanced()` [10 points]

4.1 Phase 1: Understand

Determine whether a binary tree satisfies the AVL height-balance condition: for every node in the tree, the heights of its left and right subtrees differ by at most one.

```
public static Boolean isHeightBalanced(MyTree t)
```

4.2 Phase 2: Abstract

The height-balance property must hold at *every* node, not just at the root. A tree where only the root appears balanced but some internal node has an imbalance is not a valid AVL tree. Therefore the check must recurse into both subtrees after verifying the local condition.

The height of each subtree is retrieved using `MyTreeOps.height()`, and the absolute difference is compared against the threshold of one.

4.3 Phase 3: Design

Algorithm 4 isHeightBalanced(t)

1: **if** t is empty **then return true**
2: **end if**
3: $hl \leftarrow$ height(t.left)
4: $hr \leftarrow$ height(t.right)
5: **if** $|hl - hr| > 1$ **then return false**
6: **end ifreturn** isHeightBalanced(t.left) **and** isHeightBalanced(t.right)

4.4 Phase 4: Analyze

- **Time complexity:** $O(n^2)$ in the worst case, because `height()` is recomputed separately at each recursive call. A production implementation would propagate the height value upward to achieve $O(n)$, but this straightforward version correctly captures the AVL invariant.
- **Space complexity:** $O(h)$ recursion stack.
- **Correctness:** Every node is individually verified; the conjunction of both subtree checks propagates balance correctness bottom-up through the entire tree.

4.5 Phase 5: Implement

```
public static boolean isHeightBalanced(MyTree t) {
    if (t.getEmpty()) {
        return true;
    }
    int leftHeight  = MyTreeOps.height(t.getLeft());
    int rightHeight = MyTreeOps.height(t.getRight());
    if (Math.abs(leftHeight - rightHeight) > 1) {
```

```
        return false;
    }
    return isHeightBalanced(t.getLeft()) && isHeightBalanced(t.
        getRight());
}
```

Listing 4: isHeightBalanced() in DAA2.java

5 Problem 5 — `insertHB()` [10 points]

5.1 Phase 1: Understand

Insert an integer n into an AVL tree while maintaining both the BST ordering property and the height-balance invariant.

```
public static MyTree insertHB(int n, MyTree t)
```

5.2 Phase 2: Abstract

The strategy is standard BST insertion combined with a local rebalancing step applied *on the way back up* the call stack. After inserting into the left subtree, `rebalanceForLeft` is called on the updated node, since the left side may have grown taller. Inserting into the right subtree symmetrically triggers `rebalanceForRight`. The tree is treated as persistent (immutable): new nodes are created rather than modifying existing ones in place.

5.3 Phase 3: Design

Algorithm 5 INSERTHB(n, t)

if t is empty **then return** new node(n, $\emptyset$, $\emptyset$)
end if
if $n \leq t$.value **then**
    newLeft $\leftarrow$ INSERTHB(n, t.left)
    $t' \leftarrow$ new node(t.value, newLeft, t.right) **return** REBALANCEFORLEFT(t')
else
    newRight $\leftarrow$ INSERTHB(n, t.right)
    $t' \leftarrow$ new node(t.value, t.left, newRight) **return** REBALANCEFORRIGHT(t')
end if

5.4 Phase 4: Analyze

- **Time complexity:** $O(\log n)$ — the traversal descends one path of length $h = O(\log n)$, and each rebalance call is $O(1)$.
- **Space complexity:** $O(\log n)$ new nodes created along the insertion path.
- **Correctness:** In an AVL tree, at most one rotation is needed per insertion. The rebalance functions perform a single or double rotation as appropriate, restoring the balance factor to the set $\{-1, 0, 1\}$ at every ancestor.

5.5 Phase 5: Implement

```
public static MyTree insertHB(int n, MyTree t) {
    if (t.getEmpty()) {
        return new MyTree(n, MyTree.emptyTree, MyTree.emptyTree)
            ;
    }
    if (n <= t.getValue()) {
```

```
        MyTree newLeft  = insertHB(n, t.getLeft());
        MyTree updated  = new MyTree(t.getValue(), newLeft, t.
           getRight());
        return rebalanceForLeft(updated);
    } else {
        MyTree newRight = insertHB(n, t.getRight());
        MyTree updated  = new MyTree(t.getValue(), t.getLeft(),
           newRight);
        return rebalanceForRight(updated);
    }
}
```

Listing 5: insertHB() in DAA2.java

6 Problem 6 — `rebalanceForLeft()` [15 points]

6.1 Phase 1: Understand

Restore the AVL height-balance property at node `t` when the left subtree may have grown one level taller than the right. If the left subtree is taller than the right by more than one, perform the appropriate rotation; otherwise, return the tree unchanged.

```
private static MyTree rebalanceForLeft(MyTree t)
```

6.2 Phase 2: Abstract

When height(left) > height(right) + 1, the imbalance must be corrected. The resolution depends on the structure of the left child:

- **Left-Left (LL) case** — the left child's left subtree is at least as tall as its right subtree ($ll \geq lr$). A single *right rotation* on the root restores balance.
- **Left-Right (LR) case** — the left child's right subtree is taller than its left ($lr > ll$). A *double rotation* is required: conceptually a left rotation on the left child followed by a right rotation on the root.

Both cases preserve the BST ordering property because the relative ordering of subtrees is maintained throughout the structural rearrangement.

6.3 Phase 3: Design

LL Case (single right rotation): Let L be the left child of t.

$$\text{new root} = L, \quad L.\text{right} = \text{new node}(t.\text{value},\ L.\text{old right},\ t.\text{right})$$

LR Case (double rotation): Let LR be the right child of L.

$$\text{new root} = LR, \quad LR.\text{left} = \text{new node}(L.\text{value},\ L.\text{left},\ LR.\text{left}), \quad LR.\text{right} = \text{new node}(t.\text{value},\ LR.\text{ri}$$

Algorithm 6 REBALANCEFORLEFT(t)

```
if t is empty then return t
end if
hl ← HEIGHT(t.left);  hr ← HEIGHT(t.right)
if hl ≤ hr + 1 then return t
end if
L ← t.left;  ll ← HEIGHT(L.left);  lr ← HEIGHT(L.right)
if ll ≥ lr then                                  ▷ LL: single right rotation return new node(L.value, L.left, new node(t.value, L.right, t.right))
else                                             ▷ LR: double rotation
    LR ← L.right
    newL ← new node(L.value, L.left, LR.left)
    newR ← new node(t.value, LR.right, t.right) return new node(LR.value, newL, newR)
end if
```

6.4 Phase 4: Analyze

- **Time complexity:** $O(1)$ — a constant number of height queries and node constructions.
- **Space complexity:** $O(1)$ — at most three new nodes are created per call.
- **Correctness:** The two cases are exhaustive and mutually exclusive. The LL rotation corrects a left-heavy imbalance caused by the left-left sub-path; the LR double rotation handles the left-right sub-path. In both cases the resulting tree height is reduced by one compared to the imbalanced input, restoring the AVL invariant at t.

6.5 Phase 5: Implement

```java
private static MyTree rebalanceForLeft(MyTree t) {
    if (t.getEmpty()) return t;
    int hl = MyTreeOps.height(t.getLeft());
    int hr = MyTreeOps.height(t.getRight());
    if (hl > hr + 1) {
        MyTree left = t.getLeft();
        int ll = MyTreeOps.height(left.getLeft());
        int lr = MyTreeOps.height(left.getRight());
        if (ll >= lr) {
            // LL Case: single right rotation
            return new MyTree(left.getValue(), left.getLeft(),
                   new MyTree(t.getValue(), left.getRight(), t.getRight()));
        } else {
            // LR Case: double rotation
            MyTree leftRight = left.getRight();
            MyTree newLeft   = new MyTree(left.getValue(),
                                          left.getLeft(), leftRight.getLeft());
            MyTree newRight  = new MyTree(t.getValue(),
                                          leftRight.getRight(), t.getRight());
            return new MyTree(leftRight.getValue(), newLeft, newRight);
        }
    }
    return t;
}
```

Listing 6: rebalanceForLeft() in DAA2.java

7 Problem 7 — `rebalanceForRight()` [15 points]

7.1 Phase 1: Understand

The symmetric counterpart of `rebalanceForLeft`: restore the AVL balance property at node `t` when the right subtree may have grown one level taller than the left.

```
private static MyTree rebalanceForRight(MyTree t)
```

7.2 Phase 2: Abstract

When height(right) > height(left) + 1, two sub-cases arise based on the structure of the right child:

- **Right-Right (RR) case** — the right child's right subtree is at least as tall as its left ($rr \geq rl$). A single *left rotation* on the root restores balance.
- **Right-Left (RL) case** — the right child's left subtree is taller ($rl > rr$). A *double rotation* (conceptually: right rotation on right child, then left rotation on root) is required.

This mirrors the logic of `rebalanceForLeft` exactly, with left and right roles swapped.

7.3 Phase 3: Design

RR Case (single left rotation): Let R be the right child of t.

$$\text{new root} = R, \quad R.\text{left} = \text{new node}(t.\text{value},\ t.\text{left},\ R.\text{old left})$$

RL Case (double rotation): Let RL be the left child of R.

$$\text{new root} = RL, \quad RL.\text{left} = \text{new node}(t.\text{value},\ t.\text{left},\ RL.\text{left}), \quad RL.\text{right} = \text{new node}(R.\text{value},\ RL.\text{rig}$$

Algorithm 7 REBALANCEFORRIGHT(t)

```
if t is empty then return t
end if
hl ← HEIGHT(t.left);  hr ← HEIGHT(t.right)
if hr ≤ hl + 1 then return t
end if
R ← t.right;  rr ← HEIGHT(R.right);  rl ← HEIGHT(R.left)
if rr ≥ rl then                                  ▷ RR: single left rotation return new node(R.value, new node(t.value, t.left, R.left), R.right)
else                                             ▷ RL: double rotation
    RL ← R.left
    newL ← new node(t.value, t.left, RL.left)
    newR ← new node(R.value, RL.right, R.right) return new node(RL.value, newL, newR)
end if
```

7.4 Phase 4: Analyze

- **Time complexity:** $O(1)$.
- **Space complexity:** $O(1)$ — at most three new nodes per call.
- **Correctness:** By symmetry with `rebalanceForLeft`, both cases are exhaustive. The RR rotation addresses right-heavy imbalance along the right-right sub-path; the RL double rotation addresses the right-left sub-path. BST ordering is preserved throughout.

7.5 Phase 5: Implement

```java
private static MyTree rebalanceForRight(MyTree t) {
    if (t.getEmpty()) return t;
    int hl = MyTreeOps.height(t.getLeft());
    int hr = MyTreeOps.height(t.getRight());
    if (hr > hl + 1) {
        MyTree right = t.getRight();
        int rl = MyTreeOps.height(right.getLeft());
        int rr = MyTreeOps.height(right.getRight());
        if (rr >= rl) {
            // RR Case: single left rotation
            return new MyTree(right.getValue(),
                   new MyTree(t.getValue(), t.getLeft(), right.getLeft()),
                   right.getRight());
        } else {
            // RL Case: double rotation
            MyTree rightLeft = right.getLeft();
            MyTree newLeft   = new MyTree(t.getValue(),
                                          t.getLeft(), rightLeft.getLeft());
            MyTree newRight  = new MyTree(right.getValue(),
                                          rightLeft.getRight(), right.getRight());
            return new MyTree(rightLeft.getValue(), newLeft, newRight);
        }
    }
    return t;
}
```

Listing 7: rebalanceForRight() in DAA2.java

8 Problem 8 — `deleteHB()` [10 points]

8.1 Phase 1: Understand

Delete value x from an AVL tree if it exists, returning a valid height-balanced BST containing all remaining elements. If x is not present, the original tree is returned unchanged.

```
public static MyTree deleteHB(MyTree t, int x)
```

8.2 Phase 2: Abstract

The algorithm uses standard BST deletion combined with post-deletion rebalancing applied on the way back up the recursion. Three main cases exist:

1. $x < t$.value: delete from the left subtree. After deletion the right subtree may now be relatively taller, so `rebalanceForRight` is called.
2. $x > t$.value: delete from the right subtree. After deletion the left subtree may now be relatively taller, so `rebalanceForLeft` is called.
3. $x = t$.value: the current node is to be removed. Three sub-cases arise:
   - Both children are empty: return the empty tree.
   - Only one child is non-empty: return that child directly.
   - Both children are non-empty: replace the current node's value with its *in-order predecessor* (the maximum value of the left subtree), recursively delete that predecessor from the left subtree, then call `rebalanceForRight` on the result.

Using the in-order predecessor (rather than successor) allows reuse of the already-implemented `max()` function from Problem 3.

8.3 Phase 3: Design

Algorithm 8 DELETEHB(t, x)

if t is empty **then return** t
end if
if $x < t$.value **then**
  $t' \leftarrow$ new node(t.value, DELETEHB(t.left, x), t.right) **return** REBALANCEFORRIGHT(t')
else if $x > t$.value **then**
  $t' \leftarrow$ new node(t.value, t.left, DELETEHB(t.right, x)) **return** REBALANCEFORLEFT(t')
else ▷ $x = t$.value
  if both children empty **then return** $\emptyset$
  end if
  if left child empty **then return** t.right
  end if
  if right child empty **then return** t.left
  end if
  $p \leftarrow$ MAX(t.left) ▷ in-order predecessor
  newLeft $\leftarrow$ DELETEHB(t.left, p)
  $t' \leftarrow$ new node(p, newLeft, t.right) **return** REBALANCEFORRIGHT(t')
end if

8.4 Phase 4: Analyze

- **Time complexity:** $O(\log n)$ — the search path has length $O(\log n)$, the in-order predecessor retrieval via `max()` costs $O(\log n)$, and each rebalance call is $O(1)$.
- **Space complexity:** $O(\log n)$ recursion stack frames.
- **Correctness:** Replacing the deleted node's value with its in-order predecessor preserves BST ordering, since the predecessor is the largest value smaller than the current node. Rebalancing propagates upward along the deletion path, restoring the AVL invariant at every ancestor node.

8.5 Phase 5: Implement

```java
public static MyTree deleteHB(MyTree t, int x) {
    if (t.getEmpty()) {
        return t;
    }
    if (x < t.getValue()) {
        MyTree newLeft  = deleteHB(t.getLeft(), x);
        MyTree updated  = new MyTree(t.getValue(), newLeft, t.getRight());
        return rebalanceForRight(updated);
    } else if (x > t.getValue()) {
        MyTree newRight = deleteHB(t.getRight(), x);
        MyTree updated  = new MyTree(t.getValue(), t.getLeft(), newRight);
```

```
        return rebalanceForLeft(updated);
    } else {
        if (t.getLeft().getEmpty() && t.getRight().getEmpty()) {
            return MyTree.emptyTree;
        } else if (t.getLeft().getEmpty()) {
            return t.getRight();
        } else if (t.getRight().getEmpty()) {
            return t.getLeft();
        } else {
            int predecessor  = max(t.getLeft());
            MyTree newLeft   = deleteHB(t.getLeft(), predecessor
               );
            MyTree updated   = new MyTree(predecessor, newLeft,
               t.getRight());
            return rebalanceForRight(updated);
        }
    }
}
```

Listing 8: deleteHB() in DAA2.java

Complexity Summary

Function	File	Time	Space	Notes
`isBST()`	DAA1	$O(n)$	$O(h)$	Visits every node once
`printDescending()`	DAA1	$O(n)$	$O(h)$	Reverse in-order traversal
`max()`	DAA1	$O(h)$	$O(h)$	$O(\log n)$ on balanced BST
`isHeightBalanced()`	DAA2	$O(n^2)$	$O(h)$	Height recomputed at each node
`insertHB()`	DAA2	$O(\log n)$	$O(\log n)$	Single or double rotation
`rebalanceForLeft()`	DAA2	$O(1)$	$O(1)$	Constant-time rotations
`rebalanceForRight()`	DAA2	$O(1)$	$O(1)$	Constant-time rotations
`deleteHB()`	DAA2	$O(\log n)$	$O(\log n)$	Uses in-order predecessor

Table 1: Time and space complexity of all implemented functions (h = tree height, n = number of nodes)

Academic Integrity Pledge

"By the name of Allah (God) Almighty, I hereby pledge and declare that I have completed this Midterm Exam independently. I have not engaged in cheating, plagiarism, or received unauthorized assistance in any form. I further declare that any use of AI tools was limited to a supportive role (such as grammar checking or debugging) and that the final solution is the product of my own intellectual effort. I understand that I will bear all consequences if I am found to have violated this academic integrity pledge."

Surabaya, 12 April 2026

Hisyam Syafa Raditya
5025241130